```cpp
// Define (p0, v0, t0), ..., (p4, v4, t4) as in the previous figure.
StateData state0(p0, v0, t0); // Defined similarly for all other states.
// Assume all the poses are in SE(3).

// Suppose we have a binary factor between p1 and p3.
// Assume relPose_13 is the relative pose measurement between p1 and p3
const auto binaryNoiseModel =
  noiseModel::Diagonal::Sigmas(Vector6::Ones());
const auto binaryFactor = std::make_shared<BetweenFactor<Pose3>>(
    p1, p3, relPose_13, binaryNoiseModel);

// Suppose we also have a prior (unary factor) on p1.
const auto p1PriorValue = Pose3();  // origin
const auto unaryNoiseModel =
  noiseModel::Diagonal::Sigmas(Vector6::Ones());
const auto unaryFactor = std::make_shared<PriorFactor<Pose3>>(
  p1, p1PriorValue, unaryNoiseModel);

// Suppose all poses are connected via WNOA factors.
auto Q_se3 = Vector6::Ones();  // Power spectral density

// Now, instead of defining WNOA factors and adding all the states into
// the graph, we want state1 and state3 to be interpolated.

// Factor graph visualization before interpolation:
//       unary
//         |
//  0 ---- 1 ---- 2 ----- 3 ----- 4
//  b      i      b       i       b
//          --- between ---

// Define the set of bordering states that will remain in the main solve.
set<StateData> borderStates = {state0, state2, state4};

// All other states will be interpolated.
set<StateData> interpStates = {state1, state3};

// Define the interpolation factors to wrap the two original factors.
// This also adds the WNOA factors to the states.
const auto binaryFactorWrapped = WnoaInterpFactor<Pose3>(
  binaryFactor, borderStates, interpStates, Q_se3);
const auto unaryFactorWrapped = WnoaInterpFactor<Pose3>(
  unaryFactor, borderStates, interpStates, Q_se3);

// Create graph.
NonlinearFactorGraph graphInterp;
graphInterp.add(binaryFactorWrapped);
graphInterp.add(unaryFactorWrapped);
```